AWS EC2 MONITORING AND VISUALIZATION

Using Amazon CloudWatch, Prometheus, Node Exporter and Grafana

1. Project Overview

This project demonstrates how to monitor an Ubuntu-based Amazon EC2 instance using AWS CloudWatch, Prometheus, Node Exporter, and Grafana. CloudWatch is used separately to view the standard EC2 monitoring metrics available in the AWS Console. Node Exporter exposes Linux system metrics, Prometheus collects and stores metrics from its configured targets, and Grafana queries Prometheus to display CPU and memory utilization in dashboard panels.

1.1 Problem Statement

Cloud-hosted servers need visibility into their performance and health. Without monitoring, it can be harder to recognize changes in CPU usage, memory utilization, network activity, or instance health. This project sets up a basic monitoring workflow and presents selected metrics in a readable form.

1.2 Project Objectives

- Use an Ubuntu EC2 instance as the server for the monitoring setup.
- View standard EC2 monitoring metrics in Amazon CloudWatch.
- Expose Linux system metrics through Node Exporter.
- Collect and store configured metrics using Prometheus.
- Connect Grafana to Prometheus and visualize CPU and memory utilization.
- Verify the status of monitoring services and configured Prometheus targets.

1.3 Scope

The implementation covers basic EC2 monitoring in CloudWatch and a separate Prometheus-to-Grafana workflow for Linux system metrics. CloudWatch metrics are viewed in the AWS Console; they are not integrated into the Grafana dashboard in this implementation. CloudWatch Agent installation and alerting are outside the current scope.

2. Tools and Technologies

Component	Role in the Project	How It Is Used
Amazon EC2	Cloud virtual server	Hosts Ubuntu Linux and the monitoring services.
Amazon CloudWatch	AWS monitoring service	Displays standard EC2 metrics such as CPUUtilization, NetworkIn, NetworkOut and StatusCheckFailed, where

		available.
Node Exporter	Linux metrics exporter	Exposes operating-system metrics for Prometheus to scrape.
Prometheus	Metrics collection and storage	Scrapes configured targets and stores time-series metrics.
Grafana	Visualization and dashboards	Queries Prometheus and displays metrics in dashboard panels.

3. System Architecture

The implementation contains two separate monitoring paths:

3.1 CloudWatch Path

Amazon EC2 → Amazon CloudWatch → AWS Console

AWS provides standard EC2 monitoring metrics that can be viewed in CloudWatch. These metrics are inspected separately in the AWS Console.

3.2 Prometheus and Grafana Path

Ubuntu EC2 → Node Exporter → Prometheus → Grafana Dashboard

Node Exporter exposes Linux system metrics. Prometheus scrapes metrics from configured targets and stores them. Grafana uses Prometheus as its data source and queries the metrics to display CPU and memory utilization.

3.3 Data Flow

- The Ubuntu EC2 instance runs the Linux operating system and monitoring services.
- Node Exporter exposes supported Linux system metrics.
- Prometheus collects metrics from its configured targets.
- PromQL queries calculate CPU and memory utilization.
- Grafana queries Prometheus and displays results in dashboard panels.

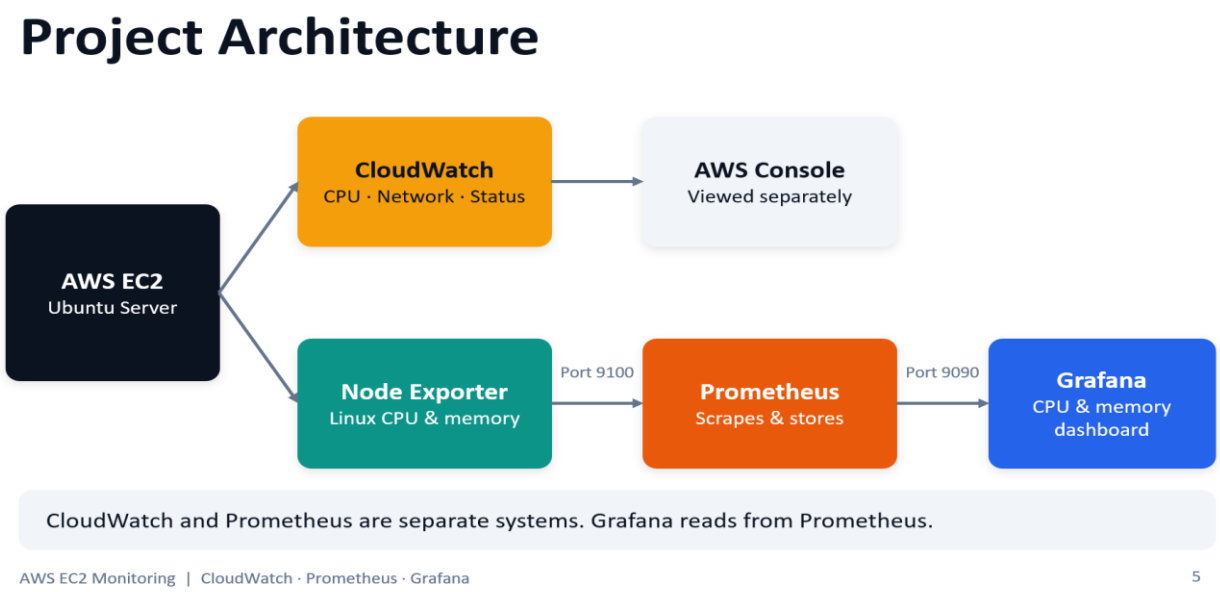


Figure. Project Architecture

4. Prerequisites and Access

- An AWS account with permission to create and manage an EC2 instance.
- An Ubuntu EC2 instance in the AWS Region selected for the project.
- SSH access to the instance.
- Security group rules restricted to the required source IP addresses.
- Prometheus, Node Exporter, and Grafana installed on the EC2 host.

Typical access ports used in this setup are SSH on port 22, Grafana on port 3000, and the Prometheus web interface on port 9090. These ports should be restricted to trusted IP addresses where possible. Node Exporter's port 9100 should not be exposed publicly.

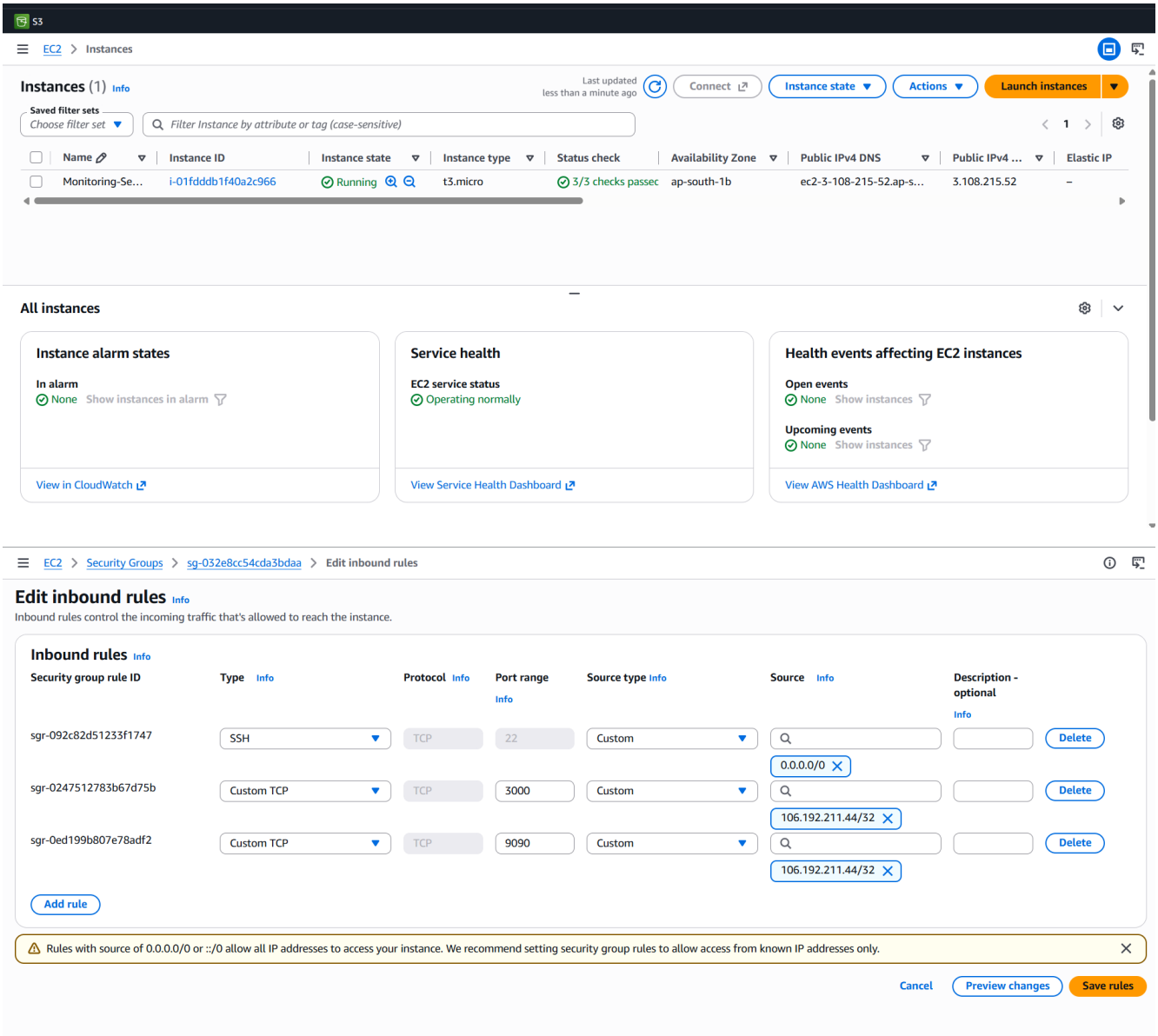


Figure. EC2 Instance and Security Group

5. Prometheus and Node Exporter Installation

The following commands install Prometheus and Node Exporter on Ubuntu using the distribution packages.

Step 1: Update package lists

```
sudo apt update
```

Step 2: Upgrade installed packages

```
sudo apt upgrade -y
```

Step 3: Install Prometheus and Node Exporter

```
sudo apt install prometheus prometheus-node-exporter -y
```

Step 4: Enable and start both services

```
sudo systemctl enable --now prometheus prometheus-node-exporter
```

Step 5: Verify Prometheus

```
sudo systemctl status prometheus
```

Step 6: Verify Node Exporter

```
sudo systemctl status prometheus-node-exporter
```

Expected result: each service should report active (running) when it is running correctly. Press q to exit the systemctl status screen.

6. Grafana Installation

Grafana was installed using the official Grafana APT repository instructions for Debian/Ubuntu. The commands below document the repository-based installation flow. If Grafana is already installed, do not rerun installation steps solely for the demonstration.

Step 1: Install prerequisite packages

```
sudo apt-get install -y apt-transport-https wget gnupg
```

Step 2: Create the APT keyrings directory

```
sudo mkdir -p /etc/apt/keyrings/
```

Step 3: Download and store the Grafana signing key

```
wget -q -O - https://apt.grafana.com/gpg.key | gpg --dearmor | sudo tee
/etc/apt/keyrings/grafana.gpg > /dev/null
```

Step 4: Add the stable Grafana repository

```
echo "deb [signed-by=/etc/apt/keyrings/grafana.gpg] https://apt.grafana.com stable main" | sudo
tee /etc/apt/sources.list.d/grafana.list
```

Step 5: Update package lists

```
sudo apt-get update
```

Step 6: Install Grafana

```
sudo apt-get install -y grafana
```

Step 7: Reload systemd and start Grafana

```
sudo systemctl daemon-reload

sudo systemctl start grafana-server
```

Step 8: Enable Grafana at boot

```
sudo systemctl enable grafana-server
```

Step 9: Verify Grafana service

```
sudo systemctl status grafana-server
```

Expected result: the Grafana service should report active (running).

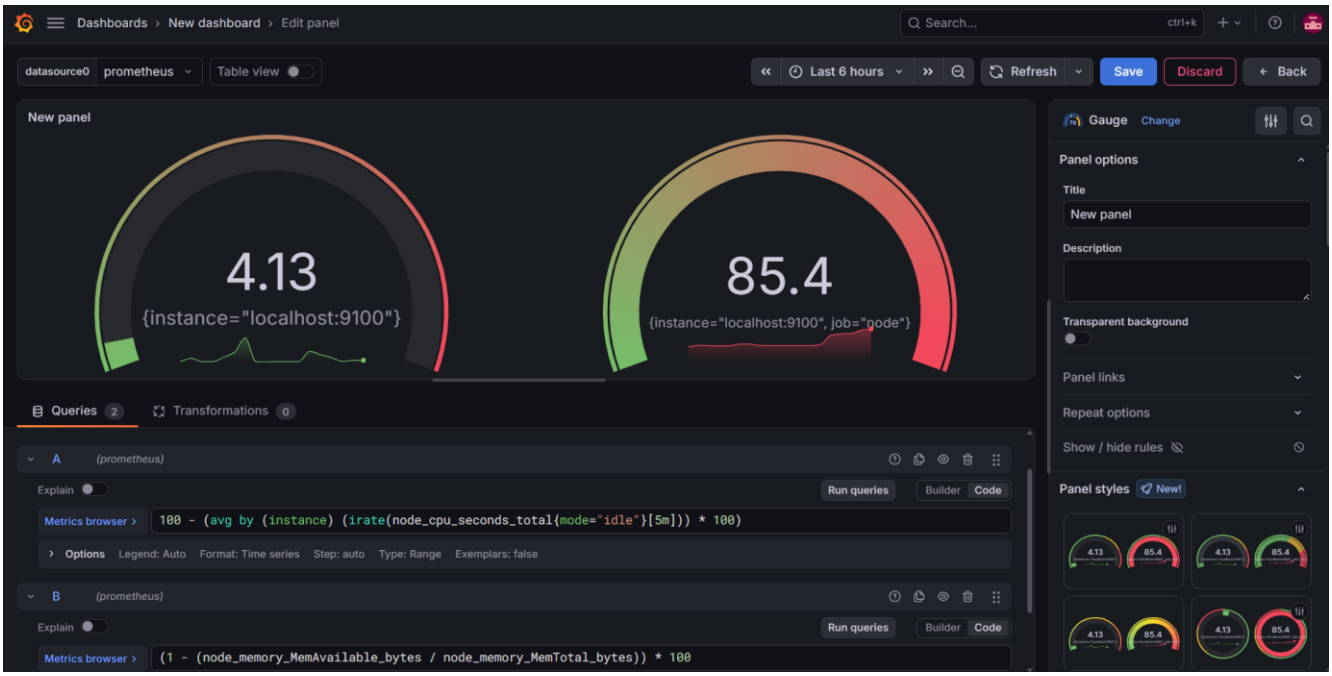
Access Grafana

```
http://YOUR-EC2-PUBLIC-IP:3000
```

Replace YOUR-EC2-PUBLIC-IP with the actual public IP address of the EC2 instance. Access depends on the instance being reachable and the security group allowing the required source IP on port 3000.

Official installation documentation: https://grafana.com/docs/grafana/latest/setup-grafana/installation/debian/

Figure. Grafana Web Interface



7. Configure Prometheus as a Grafana Data Source

Grafana must know where Prometheus is running before it can query Prometheus metrics.

Data source type: Prometheus

Prometheus URL:

```
http://localhost:9090
```

The localhost URL is appropriate when Grafana and Prometheus run on the same EC2 host and Grafana can reach Prometheus through localhost. Save the data source and use the available connection test if shown in the Grafana version.

8. Prometheus Queries (PromQL)

PromQL is Prometheus Query Language. It is used to query time-series metrics and calculate values that can be displayed in Prometheus or Grafana.

8.1 CPU Utilization

Purpose: Estimate CPU utilization as a percentage by calculating the average idle CPU rate and subtracting it from 100%.

```
100 - (avg by (instance) (irate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)
```

Explanation:

- node_cpu_seconds_total is a cumulative CPU-time metric exposed by Node Exporter.
- {mode="idle"} selects the idle CPU-time series.
- [5m] specifies a five-minute range of samples for the rate calculation.
- irate() estimates the recent per-second rate of change using the latest two samples in the range.
- avg by (instance) averages the idle rate across CPU series for each instance.
- Multiplying by 100 expresses the idle rate as a percentage; subtracting it from 100 estimates CPU utilization.

Example: if average idle CPU is approximately 70%, estimated CPU utilization is 100% − 70% = 30%.

8.2 Memory Utilization

Purpose: Calculate memory utilization as a percentage using total and available system memory.

```
(1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)) * 100
```

Explanation:

- node_memory_MemAvailable_bytes reports an estimate of memory available for starting new applications, in bytes.
- node_memory_MemTotal_bytes reports total physical memory, in bytes.
- Available memory divided by total memory gives the available proportion.
- Subtracting that proportion from 1 gives the proportion in use.
- Multiplying by 100 converts the result to a percentage.

Example: if 25% of memory is available, the calculated memory utilization is approximately 75%.

Note: These are calculated utilization metrics. Actual values depend on the live server state and the metrics available from Node Exporter.

9. Prometheus Target Verification

Prometheus targets are the configured endpoints from which Prometheus scrapes metrics.

Verification procedure:

- Open the Prometheus web interface at http://YOUR-EC2-PUBLIC-IP:9090, subject to the applicable access rules.
- Navigate to Status → Targets.
- Check whether the expected target is UP.

UP generally indicates that Prometheus can scrape the target successfully. DOWN indicates a scrape problem that may require checking service status, target configuration, connectivity, and port access.

10. Results and Validation

The project is considered successfully demonstrated when the following checks work:

- The EC2 Ubuntu instance is running and reachable.
- Prometheus, Node Exporter, and Grafana services are running.
- Prometheus shows the expected configured target as UP.
- Prometheus can return data for the CPU and memory PromQL queries.
- Grafana is connected to Prometheus as its data source.
- The Grafana dashboard displays CPU and memory utilization panels.
- CloudWatch displays available standard EC2 monitoring metrics separately in the AWS Console.

Figure. Grafana Monitoring Dashboard

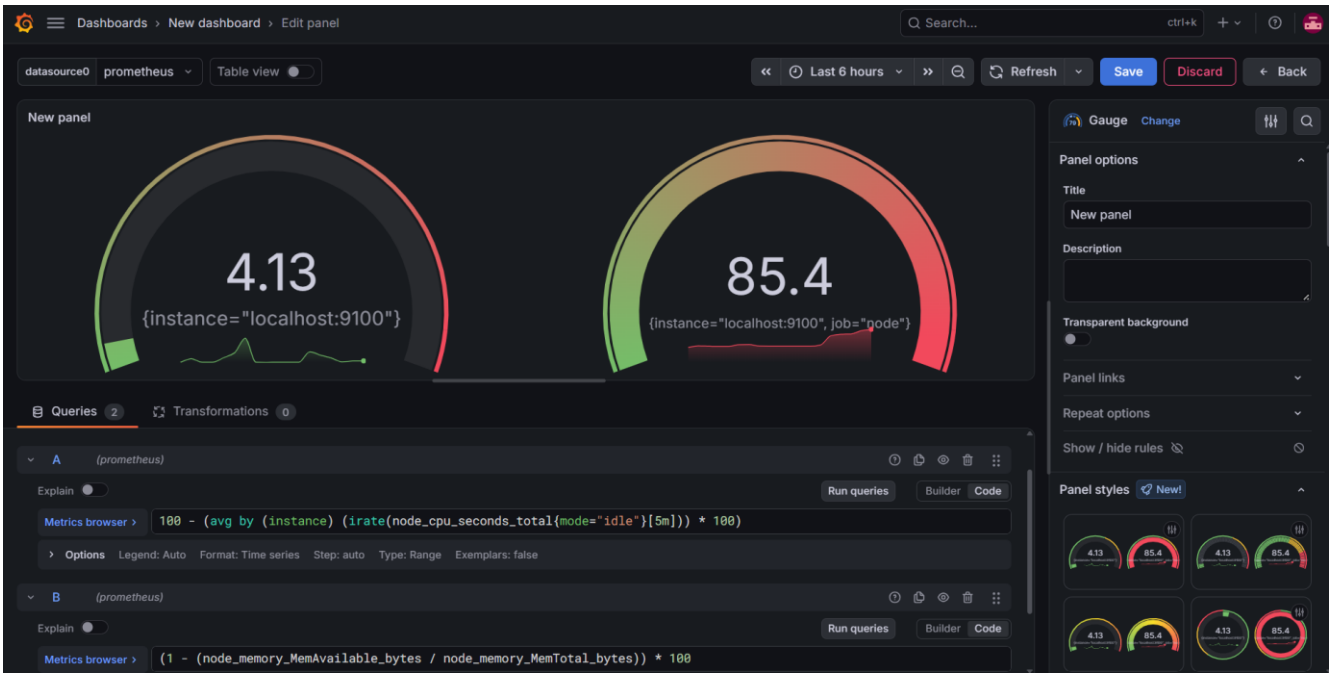
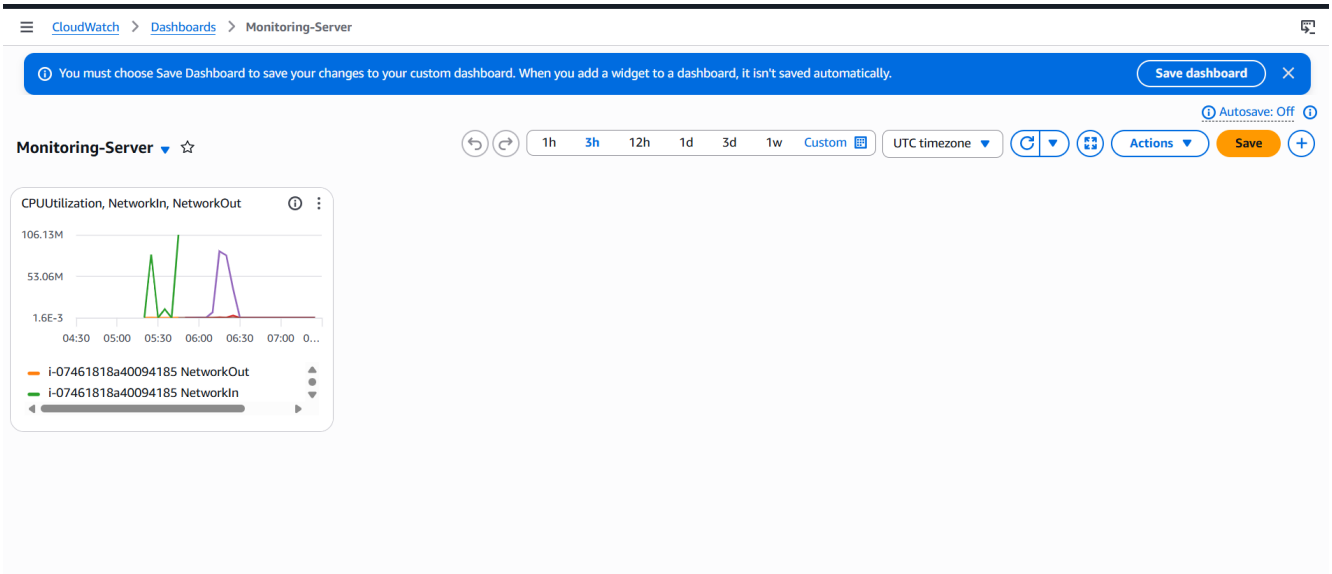


Figure. CloudWatch EC2 Metrics



11. Basic Troubleshooting and Security

11.1 If a service is not running

Check the relevant service status:

```
sudo systemctl status prometheus

sudo systemctl status prometheus-node-exporter

sudo systemctl status grafana-server
```

11.2 If Prometheus shows no data

- Check Status → Targets in Prometheus.
- Confirm Node Exporter is running.
- Confirm the target is correctly configured in Prometheus.
- Check connectivity and the relevant local service ports.

11.3 If Grafana shows no data

- Confirm the Prometheus data source URL is correct.
- Use the data source test if available.
- Confirm the Prometheus target is UP and the query returns data.

11.4 Security considerations

- Allow SSH access only from trusted IP addresses where practical.
- Restrict Grafana and Prometheus web access to required source IP addresses.

- Do not expose Node Exporter's port 9100 publicly without a specific, secured requirement.
- Use strong credentials and avoid sharing secret keys or passwords in screenshots or documentation.

12. Limitations and Future Enhancements

Current scope and limitations:

- CloudWatch metrics are viewed separately and are not integrated into the Grafana dashboard.
- No CloudWatch Agent was installed as part of this implementation.
- Alerting and automated notifications were not configured.
- The dashboard focuses on selected CPU and memory metrics.

Possible future enhancements:

- Integrate CloudWatch as an additional Grafana data source.
- Configure alert rules and notification channels.
- Add more system metrics and application-level monitoring.
- Automate installation and configuration for repeatable deployment.

13. Conclusion

This project demonstrates a basic infrastructure monitoring workflow on AWS. CloudWatch provides visibility into standard EC2 metrics, while Node Exporter and Prometheus provide Linux system metrics for querying. Grafana then visualizes selected metrics in a dashboard. The implementation provides practical experience with EC2, Linux service management, PromQL, data source configuration, and monitoring dashboards.

References

Grafana official installation documentation:

https://grafana.com/docs/grafana/latest/setup-grafana/installation/debian/

Prometheus documentation:

https://prometheus.io/docs/introduction/overview/

Amazon CloudWatch documentation:

https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/WhatIsCloudWatch.html

Amazon EC2 documentation:

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/concepts.html

Project Created & Documented By
Niketan Sarode.